When DCB Actually Helps With Correctness

When DCB actually helps — and when it's overkill

DomainSpec v2 research — 2026-06-22

For someone who knows DCB. You don't need to know our spec system — I'll give you the 90 seconds you need.

Draft, not published. Bounded toy proof, not production. I'll mark what we built and what we only argued.

A number that moved

Here is a thing that happened, before any of this was about DCB.

I worked on a marketing team. We ran an attribution model. The model runs daily and tells you how much revenue each ad channel brought in. On the 13th, it runs and says: on the 12th, this channel made R$100k. On the 14th, the same model runs again and restates the 12th — now it says R$75k. Same day, different number. The model learns more and revises the past.

Now suppose someone read the R$100k and approved a budget increase on it. The approval is real. The number it was based on is gone. Nobody lied. The fact just moved after the decision read it.

If you check "is R$100k a valid number?" the check passes. R$100k was a valid number. That is not the question. The question is whether the decision is still standing on the same ground it was built on. It isn't.

That gap is what I mean by **freshness**. Not "is this fact correct." It's "did any fact this decision read change after it read it." A static check can't see it, because a static check looks at one snapshot and asks if that snapshot is well-formed. Freshness is about two points in time: when the decision read the facts, and now.

What this has to do with DCB

You already know the machinery. A decision records a read set, with tags. It records a marker. Before you commit, you check the append condition: did anything matching those tags change after the marker. If yes, you don't commit.

That is exactly the shape freshness needs. Take the read set and the append condition off the write path, and put them on the *acceptance* of a claim. The attribution case becomes: the approval read `channel:X / metric:attributed-revenue / day:12`, at marker m. Before the budget actually ships, check whether that fact moved since m. It did — R$100k to R$75k. Reject. Re-approve against the current number.

So far this is just DCB pointed at a different decision. The interesting part is where it stops being worth it.

The system, in 90 seconds

We have a spec system. Call it DomainSpec. It is not event-sourced — that's why the DCB question is live.

A feature's behavior is spread across several artifacts, not one file. There are specs that declare concepts, rules, operations, events. There are tags in the code that point back to those spec concepts. There are validators that compare the docs against the code and flag

mismatches. There are generated views — read-models, except rebuilt from the source files instead of folded from a log.

The validators are good. They answer one question well: *is this snapshot well-formed right now.* They do not answer the freshness question. They look at one point in time. The R$100k problem lives between two points in time, and that is the part nobody owns.

That's the whole reason we looked at DCB.

The finding that surprised us

We expected DCB to be the answer for correctness. We ran a lane that did the opposite — held the problem fixed and tried to solve it *without* DCB. It changed our minds.

For most facts, DCB is too much machinery. Two cheaper things cover the common case:

- **Hash the facts you read.** Take the content of every fact the claim depended on, hash it, store the hash. Recompute at acceptance. Different hash means something moved. No event log, no marker model. Catches a change-and-revert that a timestamp would miss.
- **Just rerun the check.** If the fact is cheap and deterministic to recompute, don't track whether it's stale. Recompute it at the moment you decide. Staleness can't happen — you always read the current value.

Most of our facts are the deterministic kind. A renamed anchor, a doc-versus-code drift, a regenerated registry. Hash them or rerun them. A read set with an append condition for those facts is just a heavier reimplementation of the validators we already have.

So where does DCB survive the cost?

On facts you **can't rerun**. Go back to the attribution number. Rerunning the model doesn't recover the R$100k — it gives you R$75k. Rerunning is the thing that *changes the answer*. You can't reproduce the value the decision was built on, so "just recompute it" doesn't save you. You need to know the decision was tied to the old number and that the number moved.

The same is true for a human approval, an LLM verdict you can't reproduce token-for-token, a paid CI run, an external receipt, two promotions racing on the same feature. These don't rerun to the same answer. For these, the read set and the append condition do real work that a hash or a rerun can't. That's the residual class. That's where DCB belongs, and basically only there.

One naming consequence, because it kept us honest: the command is not `AcceptCorrectnessClaim`. It's `AcceptFreshnessClaim`. DCB doesn't decide whether the thing is correct. Your test or your reviewer or your proof decides that. DCB decides whether that decision is still standing on current facts.

The toy we actually ran

I don't want to argue this only on a story about marketing. We built a small thing.

A four-step order saga: create order, authorize payment, reserve inventory, create shipment, with compensations and policies. We wrote it in TypeScript as a bounded witness — it walks specific traces. It is not a proof. I'm calling it a witness on purpose.

We put one non-rerunnable fact in the middle: a human waiver. The waiver lets shipping proceed after the payment provider times out, as long as an external receipt exists. It's a human decision at a point in time. You can't rerun it.

The run did two separate things, and keeping them separate is the whole point:

1. **The behavioral side passed.** Payment timed out, the waiver let it continue, inventory reserved, shipment failed, and the saga compensated the completed steps instead of reporting success. The assertions held.
2. **The freshness side rejected.** The read set had seven facts. Five were deterministic — guarded by a per-fact content hash. Two were not — the human waiver and the external receipt, guarded by DCB. We then advanced the waiver from v1 to v2 after the marker. Acceptance was rejected: a fact this claim depended on, one you can't rerun and a validator would never re-check, had moved.

The saga was fine. The *acceptance* of the saga as current evidence was not. That's the only place DCB earned its keep in the whole exercise — the two facts out of seven that you can't rerun.

What I'm not claiming

- The saga is a bounded witness. It walks traces. It does not cover the state space. Don't read it as a proof.
- All of this is a toy. We built no event store, replaced no validator, changed no templates.
- DCB held up for freshness on non-rerunnable facts. It did not hold up as a way to prove behavior, and we didn't try to make it one.
- I'm hedging on the big version of this — "correctness everywhere should work this way." We have one toy and one lived case. That's not a law.

If you want the short version

DCB doesn't prove correctness. Your tests and reviewers do that. What DCB adds is a freshness guard on a decision: it reuses the read set and append condition to reject a claim when a fact it read has moved since it read it. For facts you can cheaply recompute, a content hash or a rerun does the same job with far less machinery — so keep DCB for the facts you *can't* rerun: human approvals, model verdicts, paid CI, external receipts, racing promotions. A small order-saga toy showed the split end to end: five deterministic facts guarded by hashes, one human waiver guarded by DCB, and acceptance rejected when the waiver moved — while the saga itself still ran fine.

Open question I don't have an answer for: a content hash rejects a cosmetic edit the same as a real change. Where the line goes between "the fact moved" and "the fact was reworded" is still unsettled.